Designing a Web Chatbot Powered by a Large Language Model

Architecture, Core Principles and Implementation

Marvin Leo

July 2026

Abstract

This report presents the design and implementation of a conversational chatbot with a web interface, built on top of a Large Language Model (LLM) accessed through an API. Beyond describing the technical build, this document aims to explain the core principles underlying such a system: how an LLM works, the notion of tokenization, the client-server architecture around a text-generation API, the role of the system prompt, the streaming mechanism, and how conversation history is managed. The project was developed in Python, using the Streamlit framework for the interface and the `google-genai` SDK to access Google's Gemini model.

Contents

1 Introduction

Large Language Models (LLMs) have deeply transformed how software can interact with natural language. It is now possible, with a few dozen lines of code, to build a conversational application capable of understanding a request, reasoning about it, and producing a coherent response – without having to train a machine learning model from scratch.

This personal project aimed to design a fully functional chatbot end to end: from receiving the user's message to displaying the model's generated response, including authentication key management, conversational context construction, and persistence of exchanges.

This report pursues a dual purpose:

1. to present the concrete outcome of the project: its features, its architecture, and the technical choices made;
2. to explain the underlying concepts that make this kind of system possible, so that the document remains understandable even to a reader unfamiliar with LLMs.

2 Core Principles

Before detailing the implementation, it is worth reviewing the conceptual mechanisms that govern how an LLM-based chatbot works. This section can be read as a self-contained, pedagogical introduction, independent of the project itself.

2.1 What is a Large Language Model?

A **Large Language Model** (LLM) is a neural network, most often based on the *Transformer* architecture (Vaswani et al., 2017), trained on very large amounts of text. Its training objective is conceptually simple: **predict the next word (or word fragment)** in a sequence of text, given everything that precedes it.

By repeating this prediction token after token, and feeding each generated token back into the context, the model produces coherent text, capable of answering questions, summarizing, translating, or reasoning about a problem – even though it was never explicitly programmed to perform these tasks. This is an emergent behavior arising from the model's scale and the diversity of its training data.

The models used in this project (Google's *Gemini* family) are so-called **proprietary, hosted** models: they do not run on the user's machine, but on the provider's infrastructure, and are accessed through a network API.

2.2 Tokenization

An LLM does not directly manipulate characters or words, but **tokens**: sub-word units drawn from a fixed vocabulary (typically a few tens of thousands of entries). For example, the word *"chatbot"* may be split into several tokens (`chat`, `bot`), while a very frequent word like *"the"* usually corresponds to a single token.

This notion has two practical consequences that matter to a developer:

- LLM API **billing** is based on the number of tokens sent and generated, not on the number of characters or requests;
- each model has a maximum **context window** (expressed in tokens), which limits how much text – conversation history included – it can take into account at any given time.

2.3 The *Stateless* Nature of LLMs and the Notion of History

A point that is often counter-intuitive for a developer discovering LLM APIs: **the model has no memory between calls**. Each request sent to the API is processed independently of the previous one, with no state kept on the server side.

To give the illusion of a continuous conversation, it is therefore the client application's responsibility to resend, with every new message, **the entire history of previous exchanges**. This is exactly the role played by the `st.session_state.messages` variable in our application (see Section 3): it accumulates every message of the session, and this complete list is retransmitted to the API on every turn of the dialogue.

This constraint also explains why the length of a conversation carries a growing cost: the longer the history, the more tokens are sent with every subsequent request.

2.4 The System Prompt

Most conversational LLM APIs distinguish the **system prompt** (or *system instruction*) from the messages of the conversation itself. The system prompt is a piece of text sent once, ahead of the conversation, that defines the model's expected overall behavior: its tone, its role, its constraints.

In our application, this field is exposed directly to the user in the interface (*"You are a helpful and concise assistant."* by default), which allows one to experiment concretely with the effect of *prompt engineering* on the model's behavior without touching the code.

2.5 Response Streaming

Generating a response token by token takes time – especially for long responses. Rather than waiting for the entire generation to finish before displaying it, modern APIs offer a **streaming** mode: the server sends tokens back as they are generated, as a continuous stream of small fragments (*chunks*).

On the client side, it is enough to iterate over this stream and update the display with every fragment received, producing the now-familiar "typewriter" effect of LLM-based chat interfaces. This mechanism greatly improves the perceived responsiveness of the application, even though the total generation time remains unchanged.

2.6 Client-API Architecture

Finally, it is worth emphasizing that the intelligence of the system does not reside in the application's code, but entirely in the model hosted by the provider. The application built here is a **client**: it constructs an HTTP request (authenticated with an API key), sends it to the remote service, and formats the response received. Figure 1 illustrates this separation of concerns.

3 Project Architecture and Implementation

3.1 Technical Stack

The project relies on three main building blocks:

- **Streamlit**: a Python framework for building an interactive web interface without writing HTML, CSS or JavaScript. Streamlit re-executes the entire script on every user interaction, preserving state through the `st.session_state` object;

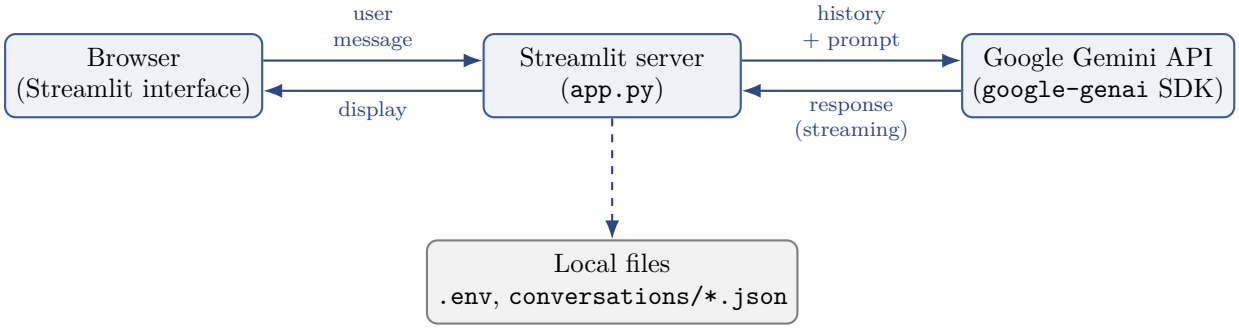

Figure 1: Overall chatbot architecture: the Streamlit server orchestrates the exchanges between the user's browser and the Gemini API, and manages local persistence (API key, saved conversations).

- **google-genai**: the official Python SDK for querying Google's Gemini models, with native streaming support;
- **python-dotenv**: loads environment variables (notably the API key) from a local, non-versioned `.env` file, so that no secret is ever exposed in the source code.

3.2 File Organization

File	Role
`app.py`	Application entry point (full chatbot logic)
`pyproject.toml`	Project dependency declaration
`.env`	Gemini API key (not versioned)
`.env.example`	Template for the `.env` file on a new environment
`conversations/`	History of saved conversations (JSON)
`lancer.bat`	One-click startup script (Windows)

3.3 Main Message-Handling Loop

The following excerpt illustrates the logical core of the application: building the conversational context from the history, calling the API in streaming mode, and progressively updating the display.

Listing 1: Handling a new user message (excerpt from `app.py`)

```
client = genai.Client(api_key=api_key)
contents = [
    types.Content(role=m["role"], parts=[types.Part(text=m["content"])])
    for m in st.session_state.messages
]

with st.chat_message("assistant"):
    placeholder = st.empty()
    full_response = ""
    stream = client.models.generate_content_stream(
        model=MODELS[model_label],
        contents=contents,
        config=types.GenerateContentConfig(system_instruction=
            system_prompt),
    )
    for chunk in stream:
```

```
            if chunk.text:
                full_response += chunk.text
                placeholder.markdown(full_response + "-")
        placeholder.markdown(full_response)
```

Two technical details are worth highlighting:

- the `contents` list is rebuilt **in full** on every message, from the entire history stored in the session – which concretely embodies the *stateless* nature of the model discussed in Section 2.3;
- the Gemini API distinguishes the roles `"user"` and `"model"` (unlike other APIs that use `"assistant"`); the application performs a display-only conversion to keep the interface consistent (see `display_role` in the appendix).

4 Features

4.1 Conversational Interface with Streaming

The user exchanges messages in a classic chat interface (user/assistant bubbles), with the response progressively displayed as it is generated.

4.2 Real-Time Configuration

The sidebar allows the user to dynamically choose:

- the Gemini model used (Flash 2.5, Pro 2.5, Flash 2.0) – each variant offering a different speed/quality/cost trade-off;
- the system prompt, to experiment with different assistant behaviors;
- the API key, automatically pre-filled from the `.env` file if available.

4.3 Saving and Reloading Conversations

A persistence feature was added to work around the ephemeral nature of Streamlit's state (lost when the tab is closed): every conversation can be saved as a timestamped JSON file in the `conversations/` folder, then reloaded later from a dropdown menu.

4.4 Simplified Startup

A `lancer.bat` script wraps the startup command (activating the virtual environment, starting the Streamlit server), enabling a one-click launch without going through a terminal.

5 Technical Choices and Rationale

5.1 Streamlit over a Conventional Web Framework

For a personal project centered on conversational logic rather than interface design, Streamlit provides a functional interface in a few dozen lines of Python, with no front-end skills required. The trade-off is a particular execution model (the entire script re-runs on every interaction), which requires thinking in terms of persistent state via `session_state`.

5.2 Choice of LLM Provider: From Claude to Gemini

The first iteration of the project relied on Anthropic's Claude API. The project was later redirected towards Google's Gemini API for cost reasons: unlike the Anthropic API, which offers no permanent free tier, Google provides a lasting free tier (with rate limits), sufficient for personal use or a portfolio demonstration.

Criterion	Anthropic API (Claude)	Google API (Gemini)
Permanent free tier	No (one-off trial credits)	Yes (with rate limits)
Credit card required	Yes	No (free tier)
Official Python SDK	`anthropic`	`google-genai`
Streaming support	Yes	Yes
Conversation roles	`user` / `assistant`	`user` / `model`

Switching providers required changing only a single file (`app.py`) and the dependency declaration, illustrating an important practical benefit: the business logic (history management, interface, persistence) is largely independent of the chosen LLM provider.

6 Result

Figure 2 shows the final interface of the application, with a test exchange illustrating streaming and the sidebar configuration.

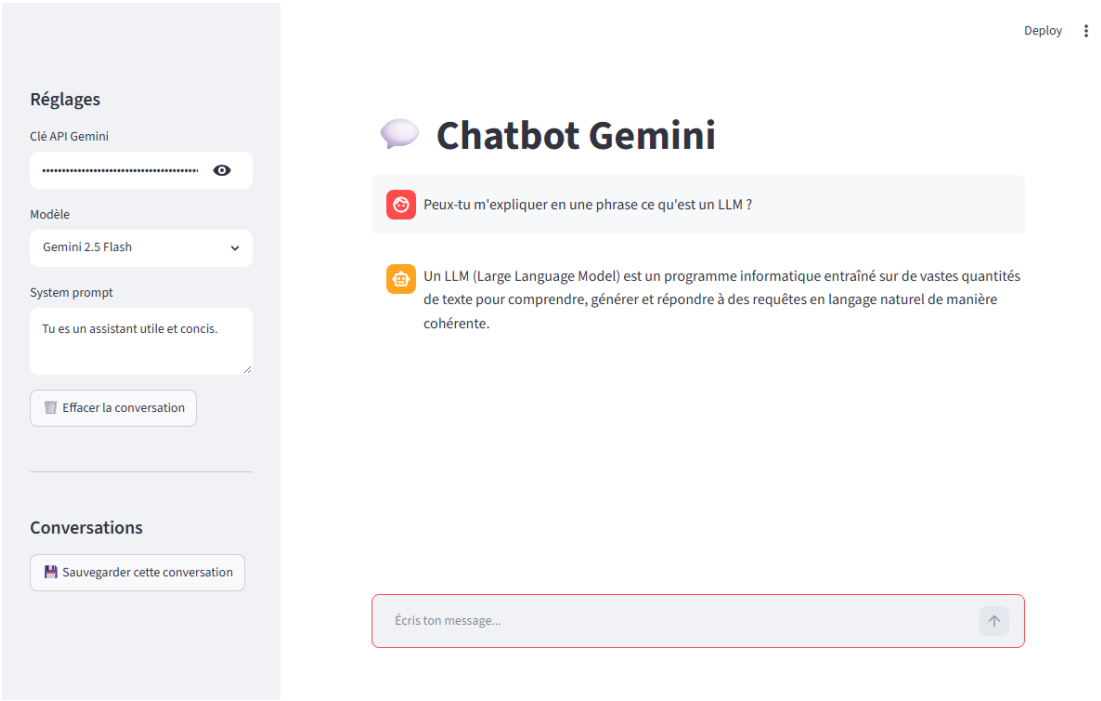

Figure 2: The chatbot interface in operation, with a test exchange and the settings panel (model, system prompt, conversation saving). The interface itself is in French, the language it was originally built and tested in.

7 Limitations and Future Directions

In its current state, the project has several acknowledged limitations, which also outline natural directions for improvement:

- **No long-term memory**: each session keeps its history only in volatile memory; a manual save is the only way to retrieve it later. A local database (SQLite) would enable automatic persistence.
- **No multi-user support**: the application is designed for local, individual use, with no authentication.
- **No document retrieval (RAG)**: the model answers solely from its training knowledge and the conversation history, with no ability to query external documents.
- **No context-length safeguard**: beyond a certain volume of exchanges, the full history sent to the API could approach the model's context window limit; a truncation or automatic summarization mechanism would be needed for heavy use.

These limitations open up concrete avenues for improvement: adding document retrieval (*Retrieval-Augmented Generation*), supporting multiple conversation profiles, or deploying the application on a publicly accessible server.

8 Conclusion

This project made it possible to put into practice, through a concrete and functional application, the full set of concepts required to integrate a large language model into an application: authentication management, conversational context construction, response streaming, and persistence of exchanges. Switching LLM providers mid-development further illustrated the portability of the chosen architecture, with the application logic remaining largely independent of the underlying model.

A Full Application Source Code (app.py)

The interface's emoji icons (speech bubble, trash bin, floppy disk, folder) have been replaced with bracketed text labels in this appendix, since the font used to typeset this document cannot display those glyphs. The user-facing strings remain in French, the language the interface was originally built and tested in.

Listing 2: Full source code of `app.py`

```python
import json
import os
from datetime import datetime
from pathlib import Path

import streamlit as st
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()

MODELS = {
    "Gemini 2.5 Flash": "gemini-2.5-flash",
    "Gemini 2.5 Pro": "gemini-2.5-pro",
    "Gemini 2.0 Flash": "gemini-2.0-flash",
}

CONVERSATIONS_DIR = Path(__file__).parent / "conversations"
CONVERSATIONS_DIR.mkdir(exist_ok=True)

st.set_page_config(page_title="Chatbot", page_icon="chat")
st.title("Chatbot Gemini")

with st.sidebar:
    st.header("Réglages")
    api_key = st.text_input(
        "Clé API Gemini",
        value=os.environ.get("GEMINI_API_KEY", ""),
        type="password",
    )
    model_label = st.selectbox("Modèle", list(MODELS.keys()))
    system_prompt = st.text_area(
        "System prompt",
        value="Tu es un assistant utile et concis.",
        height=100,
    )
    if st.button("[Effacer] Effacer la conversation"):
        st.session_state.messages = []
        st.rerun()

    st.divider()
    st.header("Conversations")

    if st.button("[Sauvegarder] Sauvegarder cette conversation"):
        if not st.session_state.get("messages"):
            st.warning("Rien à sauvegarder pour l'instant.")
        else:
            filename = f"{datetime.now():%Y-%m-%d_%H-%M-%S}.json"
            (CONVERSATIONS_DIR / filename).write_text(
```

```
                json.dumps(st.session_state.messages, ensure_ascii=False, indent=2),
                encoding="utf-8",
            )
            st.success(f"Sauvegardée sous {filename}")

    saved_files = sorted((f.name for f in CONVERSATIONS_DIR.glob("*.json")), reverse=True)
    if saved_files:
        selected_file = st.selectbox("Conversations sauvegardées", saved_files)
        if st.button("[Charger] Charger cette conversation"):
            st.session_state.messages = json.loads(
                (CONVERSATIONS_DIR / selected_file).read_text(encoding="utf-8")
            )
            st.rerun()

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    display_role = "assistant" if message["role"] == "model" else message["role"]
    with st.chat_message(display_role):
        st.markdown(message["content"])

prompt = st.chat_input("Écris ton message...")

if prompt:
    if not api_key:
        st.error("Merci de renseigner une clé API Gemini dans la barre latérale.")
        st.stop()

    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    client = genai.Client(api_key=api_key)
    contents = [
        types.Content(role=m["role"], parts=[types.Part(text=m["content"])])
        for m in st.session_state.messages
    ]

    with st.chat_message("assistant"):
        placeholder = st.empty()
        full_response = ""
        stream = client.models.generate_content_stream(
            model=MODELS[model_label],
            contents=contents,
            config=types.GenerateContentConfig(system_instruction=system_prompt),
        )
        for chunk in stream:
            if chunk.text:
```

```
100                 full_response += chunk.text
101                 placeholder.markdown(full_response + "|")
102         placeholder.markdown(full_response)
103 
104     st.session_state.messages.append({"role": "model", "content":
        full_response})
```